



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

Proyecto Antispam

Curso: SI784 – Calidad y Pruebas de Software

Integrantes:

- Jahuira Pilco, Dayan Elvis (2022075749)
- Mamani Cori, Cristhian Carlos (2023077282)

Tacna – Perú

2026

Sistema Antispam **Informe de Proyecto** Versión 1.0

CONTROL DE VERSIONES

Versión	Hecha por	Fecha	Motivo
1.0	Cristhian M.	25/06/2026	Versión original — cierre de proyecto de Unidad 3

ÍNDICE GENERAL

1. RESUMEN EJECUTIVO
 2. ALCANCE FINAL IMPLEMENTADO
 3. ARQUITECTURA Y DESPLIEGUE
 4. METODOLOGÍA DE TRABAJO
 5. CALIDAD Y PRUEBAS
 6. RIESGOS MATERIALIZADOS Y GESTIÓN
 7. LECCIONES APRENDIDAS
 8. CONCLUSIONES
 9. RECOMENDACIONES Y TRABAJO FUTURO
 10. ANEXOS
-

1. RESUMEN EJECUTIVO

Aegis Filter es un sistema antispam que centraliza la detección de contenido no deseado (spam, enlaces excesivos, palabras prohibidas) en un único motor de análisis (**Aegis Core**, backend Laravel 11), reutilizado por **cinco canales de entrada** distintos: el foro web propio, un bot de Telegram, una Alexa Skill, un plugin de WordPress y un bot de Discord. En lugar de reimplementar la lógica antispam en cada canal, cada integración delega el análisis al Aegis Core mediante su API HTTP, autenticada por canal con **Integration Keys** (credenciales hash SHA-256, emitidas y revocables desde el panel de administración).

El proyecto cumple el objetivo planteado en el FD01 (factibilidad) y el FD02 (visión): un sistema desacoplado, desplegable en contenedores, capaz de proteger múltiples superficies de contacto con el público sin duplicar la regla de negocio.

2. ALCANCE FINAL IMPLEMENTADO

Canal	Estado	Mecanismo de integración
Foro Web	✓ Completo	Formulario propio → <code>CommentController</code>
Telegram	✓ Completo	Webhook HTTPS → <code>TelegramController</code> (borra mensajes spam y advierte en el chat)
Alexa Skill	✓ Completo	Microservicio <code>alexa-bridge/</code> (Python/FastAPI) que valida la firma de Amazon y llama a <code>/api/analyze</code>
WordPress	✓ Completo	Plugin <code>wordpress-plugin/aegis-filter/</code> (hook <code>pre_comment_approved</code> , mismo patrón que Akismet)
Discord	✓ Completo	Microservicio <code>discord-bridge/</code> (Python/discord.py) conectado al Gateway por WebSocket; borra mensajes y advierte en el canal

Panel de administración (autenticado): gestión de lista negra de palabras, emisión/revocación de Integration Keys por canal, configuración del límite de URLs, bitácora de auditoría (`AnalysisLog`) con filtros por canal y resultado, y aprobación/eliminación manual de comentarios del foro.

Corrección de producción: durante el despliegue se detectó y corrigió un problema de contenido mixto (formulario de login servido por HTTP detrás del proxy TLS de Caddy); se resolvió forzando el esquema `https` en las URLs generadas por Laravel cuando `APP_URL` lo indica (`URL::forceScheme()`), verificado con un login real en producción.

3. ARQUITECTURA Y DESPLIEGUE

El detalle completo de vistas (casos de uso, lógica, implementación, procesos, despliegue) está en [FD04-Informe-Arquitectura.md](#). En resumen:

- **Aegis Core:** Laravel 11 + PHP 8.2 sobre Apache, MySQL 8 como persistencia.
- **Bridges desacoplados:** `alexa-bridge` (FastAPI, expone HTTP) y `discord-bridge` (`discord.py`, sin servidor HTTP — solo conexión saliente al Gateway), ambos en Python, mismo patrón de Dockerfile multi-stage y pruebas con `pytest` + `respx`.
- **Infraestructura:** Azure (aprovisionado con Terraform) + Docker Compose + Caddy como reverse proxy con TLS automático (Let's Encrypt) para los dominios públicos (`aegis-filter.sytes.net` , `alexa.sytes.net`).
- **Base de datos:** documentada en [Diccionario-de-Datos.md](#) — 7 tablas de dominio, sin acoplar las tablas operacionales (`comments` , `analysis_logs` , `telegram_messages`) a relaciones referenciales innecesarias; se agrupan por `channel`.

4. METODOLOGÍA DE TRABAJO

- **Control de versiones:** Git/GitHub, rama `main` estable; trabajo de riesgo medio-alto (cambios al pipeline de CI compartido) en ramas dedicadas con Pull Request antes de integrar a `main`.

- **Convenciones de código:** formalizadas en [Estandar-de-Programacion.md](#) — PSR-12 vía Laravel Pint en el backend, PEP 8 en los microservicios Python, WordPress Coding Standards en el plugin.
- **Verificación antes de producción:** cada integración nueva (WordPress, Discord) se validó primero contra una instancia local descartable (WordPress + MySQL + wp-cli en contenedores efímeros; servidor de Discord de prueba) antes de tocar el entorno productivo — ninguna credencial de producción se generó como parte de pruebas.

5. CALIDAD Y PRUEBAS

- **Pruebas unitarias e de integración:** PHPUnit (`src/tests/`), cubriendo el motor antispam (`SpamFilterTest`), el webhook de Telegram, autenticación, panel de administración y rate limiting — 45 pruebas, todas en verde.
- **Pruebas de los microservicios Python:** `pytest` + `respx` en `alexa-bridge/tests/` y `discord-bridge/tests/` , mockeando tanto la API de Aegis Core como los objetos del SDK externo (`Alexa/discord.py`) — sin depender de credenciales reales.
- **Pipeline de CI consolidado:** un único flujo de GitHub Actions (`ci.yml`) que reemplaza dos workflows previos sin gate real; ejecuta PHPUnit con cobertura, *mutation testing* (Infection), SonarCloud, Semgrep (con anotaciones SARIF) y Snyk (activable en cuanto se configure su token), publicando los reportes en GitHub Pages.
- **Análisis estático:** SonarCloud activo desde antes de esta entrega; Semgrep incorporado en esta iteración.

Pendiente explícitamente fuera de esta entrega (ver sección 9): pruebas de interfaz de usuario (UI/E2E) y BDD.

6. RIESGOS MATERIALIZADOS Y GESTIÓN

Riesgo	¿Se materializó?	Mitigación aplicada
Contenido mixto HTTP/HTTPS detrás de proxy TLS	Sí	<code>URL::forceScheme()</code> condicionado a <code>APP_URL</code> , validado con login real
Generación accidental de credenciales de producción durante pruebas	Evitado (bloqueo preventivamente)	Toda prueba de integración nueva se validó en una instancia local descartable primero
Pipeline de CI sin gate real (<i>continue-on-error</i> generalizado)	Sí, detectado en esta entrega	Rediseño completo del workflow sin supresión de errores
Conflicto de dependencias PHP al introducir <i>mutation testing</i> (Infection vs. Pint/php-parser)	Sí	Ajuste de versión de Infection a una compatible, verificado con instalación limpia

7. LECCIONES APRENDIDAS

- Centralizar la lógica antispam en un único servicio (`SpamFilterService`) y exponerla por API hizo que agregar Discord y WordPress fuera un cambio de bajo riesgo: ningún canal nuevo requirió tocar la lógica de detección, solo un adaptador de protocolo.
- Discord no ofrece *webhooks* para mensajes de chat normales (a diferencia de Telegram); cualquier bot de moderación en tiempo real necesita un proceso de larga duración conectado al Gateway, lo que cambia el modelo de despliegue (sin puerto público) respecto a un microservicio HTTP tradicional.
- Verificar localmente antes de tocar producción — incluso para cambios "solo de documentación" o de configuración — sigue siendo más rápido que depurar directamente en el entorno productivo.

8. CONCLUSIONES

- El sistema cumple el objetivo planteado: un motor antispam único, reutilizado por 5 canales heterogéneos sin duplicar lógica de negocio.
- La arquitectura desacoplada (Integration Keys + API HTTP) permitió incorporar nuevos canales (WordPress, Discord) sin modificar el núcleo Laravel, validando la decisión arquitectónica original.
- La calidad del pipeline de CI previo a esta entrega era insuficiente como gate real (errores silenciados); quedó corregido y ampliado con mutation testing y SAST.

9. RECOMENDACIONES Y TRABAJO FUTURO

- Implementar pruebas de interfaz (UI/E2E) y BDD, explícitamente diferidas para una iteración posterior por decisión del equipo.
- Elevar gradualmente el umbral mínimo de *Mutation Score* en Infection una vez que se conozca la línea base real (actualmente sin umbral estricto, primera ejecución).
- Configurar el secret `SNYK_TOKEN` y el origen de GitHub Pages como "GitHub Actions" en la configuración del repositorio (únicos pasos manuales pendientes, fuera del alcance de automatización del agente).
- Considerar slash commands en el bot de Discord y restricción de moderación por canal específico, identificados como extensiones futuras en el README del `discord-bridge`.

10. ANEXOS

- [FD01-Informe-Factibilidad.md](#)
- [FD02-Informe-Vision.md](#)
- [FD03-Informe-Requerimientos.md](#)
- [FD04-Informe-Arquitectura.md](#)
- [Diccionario-de-Datos.md](#)
- [Estandar-de-Programacion.md](#)
- [DEPLOYMENT.md](#) — pasos de despliegue de los bridges en el VPS
- [discord-bridge/README.md](#), [wordpress-plugin/aegis-filter/README.md](#) — guías de configuración por canal